Índice

Objetivos de la sesión
Conocimientos y materiales necesarios
1. Tipos de detenciones
2. Reducción de las detenciones por dependencias de datos
  2.1. Planificación de instrucciones
  2.2. Rutas de reenvío (*forwarding*)
  2.3. Renombrado de registros
3. Reducción de las detenciones por dependencias de control
  3.1. Predictor *siempre no tomado*
  3.2. Predictor dinámico de dos bits
4. Ejercicios
  4.1. Sin desenrollado de bucles
  4.2. Con desenrollado de bucles

Conceptos avanzados del *pipeline*

Área de Arquitectura y Tecnología de Computadores – Versión 1.2.12009, 25/09/2023

Objetivos de la sesión

En esta sesión se introducen conceptos avanzados de la segmentación encaminados a reducir las detenciones por riesgos de datos y las detenciones por riesgos de control. Todos ellos se ilustran sobre el simulador pyMIPS64.

Conocimientos y materiales necesarios

Para poder realizar esta sesión, el alumno debe:

- Repasar los apuntes de teoría correspondientes al tema dedicado a la CPU. Es importante además tener claras las ideas teóricas sobre el *pipeline*.
- Durante la sesión se plantearán una serie de preguntas que puedes responder en el correspondiente cuestionario del campus virtual. Puedes abrir el cuestionario en otra pestaña del navegador pinchando en el enlace mientras mantienes pulsada la tecla `Ctrl`.

1. Tipos de detenciones

El *pipeline* de la microarquitectura MIPS64 propuesta puede sufrir diferentes tipos de detenciones. La representación de las mismas en el simulador se muestra entre paréntesis:

- Por dependencia de datos RAW (RAW). Una instrucción debe detenerse mientras sus operandos fuente no están disponibles.
- Por dependencia de datos WAW (WAW). Dos instrucciones que escriben en el mismo registro deben terminar en orden. Las detenciones de este tipo sólo pueden producirse cuando se dispone de unidades de ejecución multiciclo.
- Estructurales (STR). Dos instrucciones requieren simultáneamente un mismo elemento hardware. Este elemento hardware puede ser una unidad de ejecución multiciclo no segmentada, o la etapa MEM por la que compiten dos instrucciones listas para ser terminadas. Las detenciones de este tipo sólo pueden producirse cuando se dispone de unidades de ejecución multiciclo.
- Control (CNT). Cuando se decodifica una instrucción de salto el *pipeline* debe detenerse hasta que dicho salto es evaluado.
- Terminación en orden, *In Order Retirement* (IOR). Cuando las instrucciones deben terminar en orden, tal como exige la gestión de excepciones precisas, puede ser necesario detener el *pipeline* en un punto para garantizar esta terminación en orden. Las detenciones de este tipo sólo pueden producirse cuando se dispone de unidades de ejecución multiciclo.
- Memoria (MEM). Los accesos a memoria pueden requerir varios ciclos de reloj. En ese caso, las etapas `IF` y `MEM` se detienen hasta que se completa el acceso a memoria. Estas detenciones se abordarán en la sesión de jerarquía de memoria.

En el apartado siguiente se practica con técnicas de mejora del rendimiento de la CPU basadas en la reducción o eliminación de las detenciones por riesgos de datos RAW y WAW. El final de la sesión práctica se centra en las técnicas de reducción o eliminación de detenciones por riesgos de control.

2. Reducción de las detenciones por dependencias de datos

De los tres tipos de dependencias de datos: RAW (*Read After Write*), WAR (*Write After Read*) y WAW (*Write After Write*), sólo las dependencias RAW pueden generar detenciones en la versión más simple del *pipeline* visto en clase. Éstas ocurren cuando una instrucción utiliza como operando fuente el operando destino de una instrucción anterior y ambas están en el *pipeline*, por lo que no puede ejecutarse esta última hasta que dicho operando destino haya sido escrito. En el caso de la microarquitectura MIPS64 estudiada, esta dependencia sólo es posible con operandos de registro; los operandos de memoria no pueden producir esta dependencia.

A continuación, veremos dos técnicas que afectan a las detenciones producidas por las dependencias de datos RAW: la planificación de instrucciones y las rutas de reenvío. Finalmente, se trabajará con el renombrado de registros para la reducción, o incluso eliminación, de las detenciones por dependencias de datos WAW.

2.1. Planificación de instrucciones

Una posible forma de reducir el impacto de las detenciones por dependencias RAW es depositando en el compilador la responsabilidad de minimizar el número de dependencias entre instrucciones cercanas, que ocasionarían el mayor impacto. Esto se consigue generando código donde las dependencias sobre registros están separadas varias instrucciones.

Vamos a verlo con el programa `2-3reordering.asm` de ejemplo. Este programa incluye una dependencia de datos de tipo RAW que provoca una detención en el *pipeline*.

```
.code
main:
    dadd  r6, r4, r2
    daddi r1, r2, 5
    dsub  r8, r3, r2
    xor   r1, r5, r3
    ld    r8, 120(r1)
    and   r8, r4, r1
```

- Descarga los archivos de la práctica.
- Abre el simulador y comprueba que no tiene habilitada ninguna mejora en la microarquitectura.
- ¿Qué instrucciones tienen dependencia de datos RAW? ¿De qué otras instrucciones dependen? ¿Qué registros crean esas dependencias? Responde en el cuestionario: pregunta 1.
- Dibuja sobre el papel o sobre una hoja de cálculo la evolución del cauce durante la ejecución del programa. ¿Cuántos ciclos de reloj requiere para su ejecución? Responde en el cuestionario: pregunta 2.
- Compruébalo con el simulador.
- ¿Cómo reordenarías el código del programa sin modificar su semántica, es decir, que siga haciendo lo mismo, para eliminar la detención por el riesgo RAW con el menor número de instrucciones recolocadas? Responde en el cuestionario: pregunta 3.
- Para comprobar que desaparece la detención modifica el programa `2-3reordering.asm`, guárdalo con el nombre `2-3reordering2.asm` y simula su ejecución en el simulador.

2.2. Rutas de reenvío (*forwarding*)

El principal problema de la planificación de instrucciones basada en software es que la responsabilidad recae en el compilador. Este debería conocer la microarquitectura subyacente y ser capaz de encontrar instrucciones independientes para introducirlas entre dos instrucciones dependientes, lo cual no es siempre posible.

Una solución más eficaz y transparente al software, ya que se implementa por hardware, es el uso de rutas de reenvío. Vamos a estudiarlas sobre un ejemplo.

El programa `2-3raw.asm`, mostrado a continuación, contiene varias instrucciones con dependencias de datos RAW.

```
.data
var1: word 34

.code
main:
    ori   r1, r0, 4
    daddi r2, r1, 1
    ori   r3, r1, 7
    ld    r4, var1(r1)
    daddi r6, r8, 100
    dadd  r6, r4, r4
    ori   r5, r4, 8
    beqz  r5, main
```

Analizaremos la ejecución del programa anterior en dos escenarios: sin rutas de reenvío y con rutas de reenvío activas.

2.2.1. Sin rutas de reenvío

En este caso las detenciones por dependencias de datos RAW incrementan el tiempo de ejecución del programa.

- ¿Qué instrucciones tienen dependencia de datos RAW? ¿De qué instrucciones dependen? ¿Qué registro crea esa dependencia? Responde en el cuestionario: pregunta 4.
- Dibuja sobre el papel o sobre una hoja de cálculo la evolución del cauce durante la ejecución del programa. Debes suponer que la microarquitectura implementa la evaluación *agresiva de saltos*. ¿Cuántos ciclos de reloj requiere para su ejecución? Responde en el cuestionario: pregunta 5.
- ¿Cuántos ciclos de detención ha sufrido el programa? Responde en el cuestionario: pregunta 6.
- La suma del número de instrucciones, más el número de ciclos de detención, más el transitorio inicial de 4 ciclos debe coincidir con el número de ciclos del programa. Haz esta comprobación y recuerda que sólo es aplicable a programas que no emplean instrucciones que requieren unidades de ejecución multiciclo.
- ¿Cuál es el CPI resultante? Responde en el cuestionario: pregunta 7.

Vamos a comprobar las respuestas utilizando el simulador.

- Abre el simulador pyMIPS64 y configura la microarquitectura para que la opción de evaluación agresiva de saltos (`Early branch evaluation`) esté activada y la opción de rutas de reenvío (`Forwarding`) esté desactivada. A continuación carga el programa `2-3raw.asm` y lleva a cabo la simulación.
- A partir de los resultados de la simulación comprueba tus respuestas anteriores. Si alguna no coincide debes repasar la simulación.

2.2.2. Con rutas de reenvío

El empleo de rutas de reenvío permite eliminar, salvo en casos puntuales, las detenciones por dependencias de datos RAW en el *pipeline* MIPS64. Debe recordarse que en la microarquitectura MIPS64 vista en las clases teóricas se plantean tres rutas de reenvío:

- Salida `EX` → Entrada `EX`. Permite llevar el valor del registro destino desde la salida de la etapa `EX` a la entrada de la etapa `EX` en el ciclo de reloj siguiente. Se emplea cuando las dos instrucciones dependientes son consecutivas y la primera no es una instrucción de carga.
- Salida `MEM` → Entrada `EX`. Permite llevar el valor del registro destino desde la salida de la etapa `MEM` a la entrada de la etapa `EX` en el ciclo de reloj siguiente. Se emplea en dos casos: cuando la primera instrucción es una instrucción de carga, o bien cuando hay una instrucción entre dos instrucciones dependientes.
- Salida `EX` → Entrada `ID`. Permite llevar el valor del registro destino desde la salida de la etapa `EX` a la entrada de la etapa `ID` en el ciclo de reloj siguiente. Se emplea cuando una instrucción de salto condicional con evaluación *agresiva de saltos* tiene una dependencia RAW con una instrucción anterior que no es de carga.

Vamos a responder a las mismas preguntas anteriores pero ahora suponiendo que las rutas de reenvío están activas.

- Dibuja sobre el papel o sobre una hoja de cálculo la evolución del cauce durante la ejecución del programa suponiendo que la microarquitectura implementa la evaluación *agresiva de saltos* y las rutas de reenvío. ¿Cuántos ciclos de reloj requiere para su ejecución? Responde en el cuestionario: pregunta 8.
- ¿Cuántos ciclos de detención ha sufrido el programa? Responde en el cuestionario: pregunta 9.
- Al no haber instrucciones con unidades de ejecución multiciclo, la suma del número de instrucciones, más el número de ciclos de detención, más el transitorio inicial de 4 ciclos debe coincidir con el número de ciclos del programa. Haz esta comprobación.
- ¿Qué rutas de reenvío se han activado? Cada ruta de reenvío debe especificarse de acuerdo al siguiente ejemplo: `Salida MEM de addi, r8, r9, 10` → `Entrada MEM de xor r5, r9, r6`. Responde en el cuestionario: pregunta 10.
- ¿Cuál es el CPI resultante? Responde en el cuestionario: pregunta 11.

De nuevo vamos a comprobar las respuestas utilizando el simulador.

- Abre el simulador pyMIPS64 y activa ahora las opcones `Forwarding` y `Early branch evaluation` en la microarquitectura. Observarás que la simulación se reinicia.
- Lleva a cabo de nuevo la simulación. A partir de los resultados de la simulación comprueba tus respuestas anteriores. Si alguna no coincide debes comparar la evolución del cauce sobre el papel y la simulación.
- Para comprobar si has respondido correctamente a la pregunta sobre las rutas de reenvío y los registros asociados, puedes activar la opción `Show instruction timeline details` en la configuración, ⚙.

2.3. Renombrado de registros

El reciclaje de registros es una técnica empleada por los compiladores para tratar de optimizar el rendimiento de los programas. Un compilador siempre que puede ubica las variables en registros en lugar de la memoria, al ser el acceso a los registros mucho más rápido. Por esta razón, cuando el compilador detecta en el programa fuente que a partir de un cierto punto la variable ya no aparece en ninguna instrucción, entiende que el registro sobre el que se almacena está disponible y lo reutiliza para otra variable. La consecuencia es la introducción de dependencias WAW y WAR que podrían evitarse.

El reciclaje de registros en programas complejos es inevitable, pues el número de registros que define la arquitectura del juego de instrucciones es muy limitado. No obstante, empleando la técnica de renombrado de registros se dispone de un número mucho mayor de registros físicos, denominados registros renombrados, que el procesador mapea automáticamente a los registros arquitectónicos.

Vamos a ver los efectos del reciclaje de registros sobre el rendimiento empleando el programa `2-3recycling.asm`. Observa a la derecha los comentarios mostrando el código fuente a partir del cual se obtiene el programa ensamblador.

```
.code

ori  r5, r0, 20   ; a = r5 = 20
ori  r1, r0, 10   ; y = r1 = 10
dmul r3, r1, r1   ; x = y * y = r1 * r1
dadd r3, r5, r5   ; b = a + a = r5 + r5
```

Este programa incluye una instrucción multiciclo de multiplicación de enteros, la cual requiere un tiempo de ejecución mucho mayor que la instrucción de suma que le sigue. El objetivo es forzar la ejecución fuera de orden para así generar una detención por dependencia de datos WAW. En particular, la multiplicación empleará la unidad multiciclo no segmentada, `EXB`, la cual se configurará con una latencia de 5 ciclos de reloj. Debe configurarse la microarquitectura con la opción `Out-of-order retirement` para permitir la terminación de instrucciones fuera de orden. Además, para reducir el número de ciclos del programa se supondrán activas las rutas de reenvío.

- ¿Qué instrucciones tienen dependencia de datos WAW o WAR? ¿Qué registro crea esa dependencia? Responde en el cuestionario: pregunta 12.
- Dibuja sobre el papel o sobre una hoja de cálculo la evolución del cauce durante la ejecución del programa. ¿Cuántos ciclos de reloj requiere para su ejecución? Responde en el cuestionario: pregunta 13.
- ¿Qué tipo de detenciones ha sufrido y de qué duración? Responde en el cuestionario: pregunta 14.
- ¿Cuál es el CPI del programa descontando el transitorio inicial? Responde en el cuestionario: pregunta 15.
- ¿Qué explicación tiene un CPI tan alto y alejado del teórico CPI=1? Responde en el cuestionario: pregunta 16.

Ahora debes comprobar las respuestas anteriores usando el simulador.

- Abre el simulador pyMIPS64. Dentro de la configuración de la microarquitectura y lleva a la unidad multiciclo no segmentada, `EXB`, la instrucción `dmul`. Configura además la latencia de esa unidad con 5 ciclos de reloj y activa las rutas de reenvío.
- Carga y ejecuta el programa en el simulador.

La detención por dependencia de datos WAW está causada por el reciclaje del registro `r3`, el cual se emplea para almacenar inicialmente la variable `x` y más tarde la variable `b`. Si en lugar de reciclar el registro `r3` se usase un registro diferente para almacenar la variable `b`, la dependencia de datos WAW desaparecería, pero en muchos casos no es posible por el gran número de registros que requieren los programas reales.

La técnica de renombrado de registros se basa en emplear un número de registros físicos mayor que el número de registros arquitectónicos. En el caso del simulador pyMIPS64 la microarquitectura simulada emplea 64 registros físicos de enteros y 64 registros físicos de coma flotante, frente a los 32 registros de cada tipo que tiene la arquitectura. En la microrquitectura empleada, para poder observar la reducción en ciclos de reloj obtenida del renombrado de registros será necesario activar la terminación fuera de orden de las instrucciones.

- Dibuja sobre el papel o sobre una hoja de cálculo la evolución del cauce durante la ejecución del programa, pero ahora suponiendo que la CPU implementa el renombrado de registros. ¿Cuántos ciclos de reloj requiere para su ejecución? Responde en el cuestionario: pregunta 17.
- ¿Cuál es el CPI del programa descontando el transitorio inicial? Responde en el cuestionario: pregunta 18.
- ¿Qué aceleración ha proporcionado en el programa anterior el empleo del renombrado de registros? Responde en el cuestionario: pregunta 19.
- Para cada una de las instrucciones del programa debes indicar en el cronograma los cambios que experimenta la tabla de renombrado en las etapas `ID` y `WB`. Debes emplear la notación habitual, `rx-->rry:n`, donde `rx` es el registro arquitectónico, `rry` el registro físico asociado y `n` el contador de instrucciones pendientes de terminar que usan el registro físico `rry` como registro fuente.
- ¿Qué registros arquitectónicos cambian de registro físico asociado? ¿Qué nuevos registros físicos tienen asociados? Responde en el cuestionario: pregunta 20.

Como de costumbre, vamos a comprobar las respuestas anteriores empleando el simulador.

- Activa las opciones `Register renaming` y `Out-of-order retirement` en el simulador. Observarás que se reinicia la simulación y aparece un nuevo panel de título `Register renaming table`. En ese panel se muestran los cambios que experimenta la tabla de renombrado respecto a su estado inicial, es decir, con los registros arquitectónicos asociados a registros físicos de mismo índice y con todos los contadores a cero.
- Ejecuta ciclo a ciclo el programa en el simulador. Observa cómo el renombrado se realiza en la etapa ID de cada instrucción (salvo cuando está detenida) y a partir de ese momento en el panel `Source code` y en la ventana principal se indica al lado de cada instrucción los registros físicos con los que trabaja.
- Observa el cronograma y los cambios que se producen en la tabla de renombrado en las etapas `ID` y `WB`. Comprueba que coinciden con los que has predicho. Los elementos de la tabla marcados en rojo se corresponden con cambios en el último ciclo de reloj. Recuerda además que en cualquier momento puede simular hacia adelante o hacia atrás si algún cambio no te ha quedado claro.
- Al final de la ejecución del programa la tabla de renombrado te indicará los cambios en la asignación de registros físicos a registros arquitectónicos. Comprueba que tu última respuesta fue correcta.
- Activa la opción de configuración del simulador `Show instruction timeline details` y observa la ruta de reenvío activa. Verás que al activar el renombrado de registros las rutas de reenvío se establecen entre los registros físicos.

En el ejemplo mostrado el reciclaje es fácilmente evitable, pues se dispone de muchos registros arquitectónicos sobrantes. Sin embargo, en un programa real, mucho más complejo y grande, podría no haber suficientes registros arquitectónicos. Incluso empleando el renombrado de registros, el número de registros físicos disponible puede no ser suficiente y podrían aparecer detenciones por dependencias de datos WAW, aunque lógicamente en un número menor que si no se emplease esta técnica de mejora del rendimiento.

La configuración empleada en la microarquitectura no es compatible con el soporte de excepciones precisas, pues al estar la opción `Out-of-order retirement` activada se impone la terminación de instrucciones en orden.

- Desactiva la opción `out-of-order retirement` para imponer la terminación de instrucciones en orden.
- Realiza la simulación completa del programa.
- ¿Cuántos ciclos de detención sufre la instrucción `dadd` en su fase de ejecución? ¿A qué son debidos? Responde en el cuestionario: pregunta 21.

Para comprobar la respuesta anterior puedes activar la opción `Show instruction timeline details` en la configuración, el simulador te indicará el tipo de detención sufrida.

3. Reducción de las detenciones por dependencias de control

Las detenciones por dependencias de control se producen cuando aparecen cambios de flujo en la ejecución secuencial de los programas. Estas dependencias suponen la detención del *pipeline* hasta que se determina cuál es la siguiente instrucción a ejecutar. Para el caso de los saltos condicionales, es posible utilizar técnicas de predicción de saltos para ejecutar de forma especulativa una de las dos ramas del salto. Para el caso de los saltos condicionales e incondicionales además se puede guardar la dirección destino de salto calculada para futuras ejecuciones de la instrucción.

En este apartado se emplearán dos predictores, el predictor *siempre no tomado* y el predictor dinámico de dos bits. Se compararán los resultados obtenidos con una configuración que no emplea predicción. En todos los casos se empleará evaluación *agresiva de saltos* y rutas de reenvío para reducir la longitud de los cronogramas del *pipeline*.

Partimos del programa `2-3branch.asm`, mostrado a continuación.

```
.code
    ori r1, r0, 2 ; r1 = 2
loop:
    beqz r1, endloop ; Branch on equal to zero
    daddi r1, r1, -1 ; r1 = r1 - 1
    j loop
endloop:
    xor r1, r1, r1 ; r1 = 0
```

- Abre en el simulador el archivo de programa `2-3branch.asm`.
- Configura adecuadamente la microarquitectura de la CPU en el simulador. Para evitar errores restaura la microarquitectura por defecto pulsando el botón [**Reset to defaults**] y a continuación activa las opciones de evaluación *agresiva de saltos* y rutas de reenvío.
- Realiza ahora una simulación completa. ¿Cuántos ciclos de reloj requiere la ejecución del programa anterior? Responde en el cuestionario: pregunta 22.
- Identifica las detenciones de control sufridas, indicando los ciclos detención de cada una de ellas y las causas de las mismas. Responde en el cuestionario: pregunta 23.
- Activa la opción `Show instruction timeline details` en la configuración del simulador. la información mostrada te permitirá comprobar si has identificado correctamente las detenciones de control.

3.1. Predictor *siempre no tomado*

En este apartado vamos a simular el mismo programa anterior, pero ahora empleando el predictor *siempre no tomado*.

- Configura `Always-not-taken` en la opción `Branch prediction` de la microarquitectura. El resto de opciones deben ser las mismas que las del apartado anterior.
- Realiza una simulación completa. ¿Cuántos ciclos de reloj requiere la ejecución del programa anterior? Responde en el cuestionario: pregunta 24. ¿Qué aceleración ha introducido el predictor con respecto al caso de no emplear predicción? Responde en el cuestionario: pregunta 25.
- ¿Cuántos aciertos y cuántos fallos ha experimentado el predictor? Recuerda que no existe predicción en los saltos incondicionales. Responde en el cuestionario: pregunta 26.
- Comprueba tu respuesta activando la opción `Show instruction timeline details` en la configuración del simulador. En cada etapa `ID` de la instrucción de salto condicional se muestra la predicción realizada y si se ha producido un acierto o fallo de predicción. La predicción realizada es siempre *Not Taken* (`NT`) y puede existir un acierto (`HIT`), o un fallo (`MISS`).
- ¿Cuántos ciclos de detención han sido causados por la isntrucción de salto incondicional? Responde en el cuestionario: pregunta 27.

3.2. Predictor dinámico de dos bits

En esta ocasión simularemos el mismo programa anterior, pero ahora empleando el predictor dinámico de dos bits.

- Configura `Two-bit dynamic` en la opción `Branch prediction` de la microarquitectura. El resto de opciones no deben modificarse.
- Realiza una simulación completa. ¿Cuántos ciclos de reloj requiere la ejecución del programa anterior? Responde en el cuestionario: pregunta 28. ¿Qué aceleración ha introducido el predictor con respecto al caso de no emplear predicción? Responde en el cuestionario: pregunta 29.
- ¿Cuántos aciertos y cuántos fallos ha experimentado el predictor? Recuerda que no existe predicción en los saltos incondicionales. Responde en el cuestionario: pregunta 30.
- Comprueba tu respuesta activando la opción `Show instruction timeline details` en la configuración del simulador. En cada etapa `ID` de la instrucción de salto condicional se muestra la predicción realizada y si se ha producido un acierto o fallo de predicción. La predicción realizada puede ser *Not Taken* (`NT`) o *Taken* (TK) y puede existir un acierto (`HIT`), o un fallo (`MISS`).
- ¿Cuántos ciclos de detención han sido causados por la isntrucción de salto incondicional? Responde en el cuestionario: pregunta 31.

El predictor dinámico emplea una tabla de predicción. En la etapa `IF` realiza una predicción y/o obtención de la dirección destino de salto basada en el estado de la tabla, mientras que en la etapa `ID` (o `MEM` para los saltos con condicionales con evaluación en etapa `MEM`) se realiza la evaluación.

- Lleva la simulación al comienzo, pulsando sobre el icono ⏮ o usando el atajo correspondiente.
- Observa que al estar activo el predictor de saltos dinámico ha aparecido un nuevo panel, de título `Branch prediction table`. Este panel muestra el estado de la tabla de predicción de saltos durante la simulación. Quizás tengas que redimensionarlo para visualizarlo correctamente. Cada entrada de la tabla tendrá tres campos: dirección (`Address`), dirección destino del salto (`Destination`) y los dos bits de estado (`Status bits`).
- Simula ciclo de reloj a ciclo de reloj hasta completar el ciclo de reloj 10. Durante el proceso analiza la predicción que se hace en la etapa `IF` cada vez que se ejecuta la instrucción de salto condicional. Asimismo, analiza el cambio que se produce en la tabla de traducción cada vez que la etapa `ID` procesa una instrucción de salto condicional o incondicional.
- Una vez se simula el ciclo 10 observarás que la instrucción de salto incondicional no produce una detención de control. ¿Cuál es la razón? Responde en el cuestionario: pregunta 32.
- Simula un ciclo más hasta completar el ciclo de reloj 11. Se produce un fallo de predicción. ¿Cuál es la razón? Responde en el cuestionario: pregunta 33.

4. Ejercicios

El compilador puede jugar un papel fundamental a la hora de generar código para evitar detenciones por riesgos de control.

Las estructuras de control que más dependencias de control introducen son los bucles. Estos están gobernados por una condición que debe ser evaluada en cada iteración, por lo que se produce al menos un riesgo de control cada vez que el bucle itera.

Para evitarlo, cuando se activan las opciones de optimización del compilador, éste puede hacer uso de una optimización denominada desenrollado de bucles. Veámoslo con un ejemplo.

```
for (i = 0; i != 16; i++)
{
    a = a + c;
}
```

Aunque el bucle mostrado podría sustituirse por la expresión `a = 16 * c`, se opta por un ejemplo sencillo para facilitar la compresión.

4.1. Sin desenrollado de bucles

El código que generaría el compilador podría ser el siguiente:

```
    .code

main:
    xor    r8, r8, r8
    daddi  r9, r0, 16
for:
    beq    r8, r9, end
    dadd   r10, r10, r11  ; a = a + c
    daddi  r8, r8, 1
    j      for
end:
    nop
```

- Edita el fichero `2-3loop.asm` para que tenga el contenido mostrado arriba.
- Abre el simulador pyMIPS64, carga el fichero `2-3loop.asm` y configúra el simulador con las rutas de reenvío activas, evaluación *agresiva de saltos* y el predictor dinámico de dos bits. A continuación ejecuta el programa. ¿Cuántos ciclos necesita para ejecutarse? Responde en el cuestionario: pregunta 34.

4.2. Con desenrollado de bucles

El desenrollado de bucles busca disminuir el número de veces que se comprueba la condición del bucle. Para ello, se replica el cuerpo del mismo en un número divisor del número total de iteraciones. En el ejemplo, se podría replicar el cuerpo del bucle 4 veces, ya que 4 es divisor del total de 16 iteraciones.

```
    .code

main:
    xor    r8, r8, r8
    daddi  r9, r0, 16
for:
    beq    r8, r9, end
    dadd   r10, r10, r11  ; a = a + c

    dadd   r10, r10, r11  ; a = a + c

    dadd   r10, r10, r11  ; a = a + c

    dadd   r10, r10, r11  ; a = a + c
    daddi  r8, r8, 4
    j      for
end:
    nop
```

Fíjate cómo la variable de control del bucle debe ser incrementada por el número de veces que se replica el cuerpo del bucle.

- Edita el fichero `2-3loop-unroll.asm` para que tenga el contenido mostrado arriba.
- Abre el simulador pyMIPS64, carga el fichero `2-3loop-unroll.asm`, configura el simulador con las mismas opciones que el caso anterior y simula el programa. ¿Cuántos ciclos necesita para ejecutarse? Responde en el cuestionario: pregunta 35.
- ¿Cuál es la aceleración de la versión con el bucle desenrollado respecto de la versión inicial? Responde en el cuestionario: pregunta 36.

Como ves, el desenrollado de bucles constituye una optimización que puede utilizar el compilador para acelerar la ejecución de programas. Como contrapartida, los programas serán más grandes en memoria, ya que estarán formados por más instrucciones.